

Program Specification: Syntax Binding XML-to-Hierarchical-CSV Conversion

1. Purpose

This document specifies the syntax binding conversion program used by the UADC Proof of Concept.

The program converts an XML invoice document into a dimension-based hierarchical CSV by applying a syntax binding CSV. During forward conversion it also writes JSON metadata that relates the CSV columns to the generated taxonomy. It can also reverse a hierarchical CSV back to XML with the same binding information.

Implementation-level processing details, including XPath parent-context recursion, **semantic_path** handling, **dInvoice** and **dInvoiceLine** dimension assignment, and function-level data flow, are consolidated in **../README_SCRIPT_PROCESSING.md**.

All paths in this document are relative to the **UADC_PoC** working directory after the repository is pushed or cloned, except paths beginning with **../**, which refer to sibling directories of **UADC_PoC**.

2. Main Program

Program:

```
src/syntax_binding.py
```

The converter is self-contained for the supported namespace and XPath helper functions used by the operational hierarchical converter.

3. Input Files

3.1 XML input

The XML input is a UBL Invoice XML document.

PoC sample:

```
samples/input/openpeppol_ubl_invoice_minimal.xml
```

Package sample:

```
../syntax_binding_revised_package/invoice.xml
```

3.2 Syntax binding CSV

The binding CSV maps semantic model paths to XML XPath expressions.

PoC binding:

```
specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv
```

Package binding:

```
../syntax_binding_revised_package/bindings.csv
```

Required operational binding columns:

```
sequence
level
structured_csv_level
type
name
datatype
multiplicity
semantic_path
structured_csv_column
element
xpath
```

Rows without both **semantic_path** and **xpath** are ignored for fact extraction. **type=C** rows define semantic class contexts and repeated row scopes. **type=A** rows define extractable or generatable values.

3.3 Binding-Embedded LHM Layout

The syntax binding CSV is defined by copying the required LHM columns and preserving the LHM row order. The converter therefore uses the binding table itself to define:

- output column order;
- BG dimension columns;
- BT value columns;
- mapping from semantic paths to final LHM **element** names;
- repeating BGs based on **multiplicity**;

- repeat context XPath for BG rows where available.

The LHM source remains the governance source for these columns:

```
specs/lhm/EN16931_CIUS_Invoice_LHM.csv
```

3.4 Template CSV

When **--template-csv** is supplied, the converter uses its header as the output column order and infers some field-to-dimension placement from non-empty template rows.

Package template:

```
../syntax_binding_revised_package/invoice.csv
```

If **--template-csv** is supplied, the binding-derived layout controls the main field order and the template may still provide field placement hints.

4. Forward Output

The output is a UTF-8-SIG CSV file.

PoC output:

```
out/phase1/openpeppol_ubl_invoice_minimal.csv
```

Package output:

```
out/phase1/<input-xml-stem>.csv
```

The output contains:

- **dInvoice** as the root dimension column;
- additional **d*** columns for repeating BGs;
- BT value columns after dimension columns;
- one root row for invoice-level facts;
- one row per repeated BG context where bindings produce values.

Forward conversion also writes JSON metadata. By default the metadata file name is:

```
<output-csv-stem>.json
```

For example:

```
out/phase1/openpeppol_ubl_invoice_minimal.json
```

The metadata is an xBRL-CSV metadata document that Arelle can load with the **loadFromOIM** plugin. It links the structured CSV table to the generated xBRL-CSV taxonomy entry points and maps CSV columns to taxonomy concepts. The report **documentInfo.taxonomy** property must not be empty; missing **en16931-oim** is treated as a conversion error. The JSON metadata names the xBRL-CSV OIM taxonomy entry point **out/taxonomy/plt/en16931-oim-.xsd**.

5. Reverse Output

When **--reverse** is used, the input is a hierarchical CSV and the output is an XML file.

Reverse output example:

```
out/reverse/en16931_reverse_invoice.xml
```

The reverse converter:

- reads the hierarchical CSV rows;
- resolves fields through the syntax binding CSV;
- uses the binding-derived layout to associate fields with dimension rows;
- creates UBL Invoice XML elements from bound XPath expressions;
- creates one XML context for each repeated dimension row that has bound values.
- derives required **currencyID** attributes for amount elements from invoice currency fields.
- keeps out-of-context absolute XPaths rooted at the document even when the semantic term belongs to a repeated class;
- writes BT-90 below **AccountingSupplierParty** and never creates a nested **Invoice** below **PaymentMeans**.

6. Command-Line Interface

```
syntax_binding.py XML_FILE -b BINDING_CSV -o OUTPUT_CSV [options]
```

Reverse form:

```
syntax_binding.py INPUT_CSV --reverse -b BINDING_CSV -o OUTPUT_XML
[options]
```

Arguments:

- **XML_FILE**: input XML file in forward mode.
- **INPUT_CSV**: input hierarchical CSV file in reverse mode.
- **-b, --binding**: syntax binding CSV.
- **-o, --output**: output hierarchical CSV in forward mode, or output XML in reverse mode.
- **--template-csv**: optional CSV template defining column order and dimension placement.
- **--metadata-output**: optional JSON metadata output path. Default is the output CSV path with **.json** extension.
- **--taxonomy-base**: generated taxonomy base directory referenced by JSON metadata. Default is **out/taxonomy**.
- **--reverse**: convert hierarchical CSV back to XML.
- **--drop-empty-columns**: remove columns that have no values in the generated rows.
- **--d-invoice**: value written into the **dInvoice** column. Default is **1**.
- **-e, --encoding**: CSV encoding. Default is **utf-8-sig**.

7. Forward Processing Model

7.1 Namespace handling

The converter collects XML namespace declarations from the input XML using **xml.etree.ElementTree.iterparse**.

XPath helper functions support namespace-prefixed elements such as:

```
/Invoice/cbc:ID
/Invoice/cac:AccountingSupplierParty/cac:Party/cac:PartyName/cbc:Name
```

7.2 Binding-Embedded LHM Layout Loading

When the syntax binding table is loaded:

1. C rows are treated as BG/class rows.
2. A C row becomes a dimension only if it is the invoice root or if its multiplicity is repeating, such as **0..*** or **1..***.
3. Non-repeating BG rows remain semantic grouping nodes and are not emitted as dimensions.
4. Dimension names are generated as **d** + UpperCamelCase(**element**).
5. A rows are treated as BT/fact rows.
6. Fact column names are taken from **structured_csv_column**.
7. Each fact is assigned to the nearest ancestor repeated dimension.

Repeating multiplicity is detected when the upper bound is:

```
*  
n  
unbounded
```

or a numeric value greater than **1**.

7.3 Binding parsing

Each binding row is converted into an internal binding record:

```
semantic_path  
xpath  
root  
dimension  
filter_field  
filter_value  
field
```

The converter supports:

- root-level paths such as **\$.invoice.invoiceNumber**;
- dimension paths with optional filter predicates;
- generic nested semantic paths resolved through the nearest LHM dimension.

7.4 Row generation

Forward conversion uses XML parent context recursion rather than processing repeated groups independently. The converter creates:

- a root invoice row for invoice-level facts;
- repeated BG rows for repeated contexts such as VAT breakdown or invoice line.

For repeated BG rows:

1. The converter builds a **BindingClass** tree from **type=C** rows.
2. The root call starts with **dInvoice=1**.
3. For the current class, direct **type=A** facts are extracted with XPath relative to the current XML context.
4. Non-repeated child classes reuse the current Structured CSV row.
5. Repeated child classes are processed one occurrence at a time under the current parent XML context.
6. Each repeated occurrence receives a 1-based dimension value, such as **dInvoiceLine=2**.
7. Ancestor repeated dimension values are copied into nested repeated rows.

A non-repeating child class is flattened into its parent row. A repeating child class is not flattened: its first and subsequent occurrences are all written on separate child rows. For example:

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,,a1V1,a2V1,,,
1,1,,,b1V1,b2V1,b3V1
1,2,,,b1V2,b2V2,b3V2
```

The parent row therefore has empty **b1**, **b2**, and **b3** fields. A row such as **1,1,a1V1,a2V1,b1V1,b2V1,b3V1** is invalid because it mixes facts owned by **dAaa** with facts owned by the repeating **dBbb** scope.

7.5 Column handling

If the LHM is used, field order is:

```
dimension columns first, then fact columns
```

If a template is used without LHM, the template header controls the output order.

If neither LHM nor template provides a header, field names are derived from bindings.

8. Metadata JSON Processing Model

In forward mode the converter writes JSON metadata after the structured CSV is generated.

The metadata structure follows the xBRL-CSV report package shape:

- **documentInfo**: xBRL-CSV document type, namespaces, and taxonomy entry point references.
- **tables**: the generated structured CSV table and its relative URL.
- **tableTemplates**: table-level dimensions and per-column concept mappings.

Dimension columns are declared as table dimensions:

```
"plt:d_en16931_Invoice": "$dInvoice"
```

Fact columns are declared with xBRL concepts:

Example:

```
{
  "InvoiceNumber": {
    "dimensions": {
      "concept": "en16931:InvoiceNumber"
```

```

    }
  }
}
```

Amount facts receive a default test unit of **iso4217:JPY** in metadata when the LHM datatype indicates an amount. Production use should derive the unit from the applicable currency term.

9. Reverse Processing Model

In reverse mode:

1. The converter reads the input hierarchical CSV with **csv.DictReader**.
2. The converter reads the binding-derived layout and binding CSV in the same way as forward mode.
3. Before XML creation, each fact is checked against the deepest populated dimension on its row. Facts owned by an ancestor dimension are rejected on a repeated child row, so a mixed parent/child row cannot be reverse-converted.
4. Root-level fields are written to XML paths directly below the invoice root.
5. Dimension fields are grouped by their **d*** dimension column.
6. For each dimension row with bound values, the converter creates a repeated XML context from the BG XPath.
7. Bound field values are written to relative XPaths under that context.
8. XPath predicates such as **cbc:ChargeIndicator=false()** and **cbc:DocumentTypeCode='130'** are materialized as child values when a matching XML context is created.
9. Amount elements receive **currencyID** from the invoice document currency field. Tax accounting currency TaxTotal paths receive **currencyID** from the tax accounting currency field.
10. If **relative_xpath(binding.xpath, repeat_path)** remains absolute, the binding is outside the repeated syntax context and is written from the document root. Only contained paths are written relative to the repeated element.
11. UBL-required supporting elements that are not EN 16931 BT values are added where needed for schema validation, including **cac:TaxScheme/cbc:ID** and missing **cbc:ChargeIndicator=false**.
12. Generated UBL child elements are normalized to UBL schema sequence. When **--ubl-schema-root** or **--ubl-schema-url** is supplied, the converter derives child order from XSD **xs:sequence** declarations. Without a schema source, it uses the built-in fallback order for the supported Invoice structures.

The reverse converter currently targets the PoC UBL Invoice binding pattern. It is intended for round-trip verification of the structured CSV representation rather than canonical XML reproduction.

10. Currency Attribute Rules

The syntax binding treats **currencyID** as syntax metadata derived from semantic currency terms:

- BT-5 **DocumentCurrencyCode** is the default **currencyID** for invoice amount elements.

- BT-6 **TaxAccountingCurrencyCode** is the **currencyID** for the tax accounting currency TaxTotal branch.
- BT-110 and BG-23 use the UBL path predicate **cbc:TaxAmount/@currencyID=/Invoice/cbc:DocumentCurrencyCode**.
- BT-111 uses the UBL path predicate **cbc:TaxAmount/@currencyID=/Invoice/cbc:TaxCurrencyCode**.

Forward conversion evaluates these absolute predicate reference paths from the document root. For **Allowance-example.xml**, BT-110 resolves to **1225.00 EUR** and BT-111 resolves to **9324.00 SEK**.

BT-90 illustrates a cross-scope reverse mapping. Its semantic path is under payment instructions/direct debit, but its UBL XPath points to **/Invoice/cac:AccountingSupplierParty/.../cbc:ID**. Reverse conversion therefore keeps that XPath document-rooted rather than constructing **PaymentMeans/Invoice/AccountingSupplierParty**.

The CSV therefore stores semantic amount values and currency code values separately. Reverse conversion writes the required UBL **currencyID** attributes.

11. Constraints

- The converter does not validate the XML against UBL schemas internally; the regression test **tests/test_roundtrip_xml_ubl_schema.py** performs UBL 2.1 schema validation.
- The converter does not validate EN 16931 business rules.
- The converter does not run Arelle internally; the regression test **tests/test_xbrl_csv_metadata_arelle.py** validates generated metadata with Arelle.
- The converter does not evaluate full XPath 2.0.
- Supported predicates are intentionally limited to the cases used by the PoC bindings.
- Template columns alone do not create rows; rows require matching bindings.
- Reverse XML element order can be derived from a supplied UBL XSD schema root or UBL Invoice schema URL. The built-in child-order table in **syntax_binding.py** is only the fallback when no schema source is supplied.
- Reverse XML may omit unbound XML content.
- Reverse XML is generated from bound CSV values and is not intended to be byte-for-byte identical to the source XML.

12. Regression Tests

PoC LHM-driven conversion test:

```
tests/test_lhm_hierarchical_csv_layout.py
```

Package template/binding conversion test:

```
tests/test_syntax_binding.py
```

OpenPeppol structured conversion test:

```
tests/test_openpeppol_invoice_conversion.py
```

Reverse conversion round-trip test:

```
tests/test_syntax_binding_reverse.py
```

Round-trip artifact and metadata test:

```
tests/test_roundtrip_artifacts.py
```

13. Non-Goals

The syntax binding converter does not:

- generate LHM definitions;
- generate XBRL taxonomy files;
- run semantic binding to flat CSV;
- perform full EN 16931 or OpenPeppol validation;
- publish generated **out/** files to GitHub.